Relational Algebra: Select

R(a_id, b_id)

a_id	b_id
a1	101
a2	102
a2	103
a3	104

$\sigma_{a\_id='a2'}(R)$

a_id	b_id
a2	102
a2	103

```
SELECT * FROM R
WHERE a_id='a2'
  AND b_id>102;
```

Relational Algebra: Projection

Generate a relation with tuples that contains only the specified attributes.

- $\rightarrow$ Rearrange attributes' ordering.
- $\rightarrow$ Remove unwanted attributes.
- $\rightarrow$ Manipulate values to create derived attributes.

Syntax: $\Pi_{A1,A2,\ldots,An}(R)$

$\Pi_{b\_id-100,a\_id}(\sigma_{a\_id='a2'}(R))$

```
SELECT b_id-100, a_id
FROM R WHERE a_id = 'a2';
```

Relational Algebra: Union

Generate a relation that contains all tuples that appear in either only one or both input relations.
Syntax: $(R \cup S)$

- Standard set algebra rules apply.

```
(SELECT * FROM R)
UNION
(SELECT * FROM S);
```

Relational Algebra: Intersection

Generate a relation that contains only the tuples that appear in both of the input relations. **Syntax:**

$(R \cap S)$

R (a1, a2, a3)
S (a3, a4, a5)
→ Result: (a3)

```
(SELECT * FROM R)
INTERSECT
(SELECT * FROM S);
```

Relational Algebra: Difference

Generate a relation that contains only the tuples that appear in the first and not the second of the input relations. **Syntax:** $(R - S)$

R (a1, a2, a3)
S (a3, a4, a5)
→ Result: (a1, a2)

```
(SELECT * FROM R)
EXCEPT
(SELECT * FROM S);
```

Relational Algebra: Product

Generate a relation that contains all possible combinations of tuples from the input relations. **Syntax:**

$(R \times S)$

```
SELECT * FROM R CROSS JOIN S;
SELECT * FROM R, S;
```

Relational Algebra: Join

Generate a relation that contains all tuples that are a combination of two tuples (one from each input relation) with a common value(s) for one or more attributes. **Syntax:** $(R \bowtie S)$

Relational Algebra: Join (Examples)

R(a_id, b_id)

a1	101
a2	102
a3	103

S(a_id, b_id, val)

a3	103	XXX
a4	104	YYY
a5	105	ZZZ

Result $(R \bowtie S)$**:**

a3	103	XXX

```
SELECT * FROM R NATURAL JOIN S;
SELECT * FROM R JOIN S USING (a_id, b_id);
SELECT * FROM R JOIN S ON R.a_id = S.a_id;
```

Relational Algebra: Extra Operators

- Rename (ρ)
- Assignment ($R \leftarrow S$)
- Duplicate Elimination (δ)
- Aggregation (γ)
- Sorting (τ)
- Division ($R \div S$)

Observation

Relational algebra defines an ordering of the high level steps of how to compute a query.

- $\rightarrow$ Example: $\sigma_{b\_id=102}(R \bowtie S)$ vs. $(R \bowtie (\sigma_{b\_id=102}(S)))$

A better approach is to state the high-level answer that you want the DBMS to compute.

- $\rightarrow$ Example: Retrieve the joined tuples from R and S where b_id equals 102.

Relational Model: Queries

The relational model is independent of any query language implementation.

- SQL is the de facto standard (many dialects).

```
for line in file:
 record = parse(line)
 if record[0] == "GZA":
  print(int(record[1]))
```

```
SELECT year FROM artists
WHERE name = 'GZA';
```

Data Models

- **Relational** ← This Course
- Key/Value
- Graph
- Document / JSON / XML / Object
- Wide-Column / Column-family
- Array (Vector, Matrix, Tensor)

Data Models

- Relational
- Key/Value
- Graph
- **Document / JSON / XML / Object** ← Leading Alternative
- Wide-Column / Column-family
- **Array (Vector, Matrix, Tensor)** ← New Hotness

Document Data Model

A collection of record documents containing a hierarchy of named field/value pairs.

- → A field's value can be either a scalar type, an array of values, or another document.
- → Modern implementations use JSON. Older systems use XML or custom object representations.

Avoid "relational-object impedance mismatch" by tightly coupling objects and database.

Document Data Model (Relational Version)

Artist $\bowtie$ **ArtistAlbum** $\bowtie$ **Album**

$R1(id, \ldots)$ $R2(artist\_id, album\_id)$ $R3(id, \ldots)$

Document Data Model (JSON vs Code)

Document / JSON

```
{
 "name": "GZA",
 "year": 1990,
 "albums": [
  {
   "name": "Liquid Swords",
   "year": 1995
  },
  ...
 ]
}
```

Application Code (Java/C++)

```
class Artist {
  int id;
  String name;
  int year;
  Album albums[];
}
```

Vector Data Model

One-dimensional arrays used for nearest-neighbor search (exact or approximate).

- → Used for semantic search on embeddings generated by ML trained transformer models (think ChatGPT).
- → Native integration with modern ML tools and APIs (e.g., LangChain, OpenAI).

At their core, these systems use specialized indexes to perform NN searches quickly.

Vector Data Model

Vector Data Model

Vector Data Model

Conclusion

- Databases are ubiquitous.
- Relational algebra defines the primitives for processing queries on a relational database.
- We will see relational algebra again when we talk about query optimization + execution.

Next Class

Modern SQL

- → Make sure you understand basic SQL before the lecture.

Ask Andy Anything

- Questions about database industry?
- Questions about database jobs?
- Questions about database systems?